

1. Title Page

Educational Practice 2025-2026

Project Track: Machine Learning

Project Topic: Spam Message Classification

Team Name: [Double Head]

2. Team Members and Contributions

Our team consists of two members. We divided the tasks equally to finish the project effectively:

[Gibrat Kaiyrzhanov]: Responsible for the technical part. Created the GitHub repository, wrote the Python code, implemented the machine learning model, created the interactive menu for manual testing, and recorded the demo video.

[Plan Nurislam]: Responsible for theory, research, and testing. Prepared the structure of the project, studied the dataset, analyzed the model's errors (testing phase), and wrote this final PDF report.

3. Project Overview

In the modern digital world, people receive thousands of messages every day. Unfortunately, many of these messages are "spam" (unwanted ads, scams, or dangerous links). The goal of our project is to create an automatic system that can read a text message and decide if it is "Spam" or "Ham" (a normal message). To do this, we used Machine Learning algorithms that learn from examples instead of using simple keywords.

4. Problem Statement

Manual filtering of spam is impossible today because there is too much information. Also, scammers constantly change their texts to trick basic filters.

The main tasks of our project were:

Find a good dataset with real examples of spam and normal messages.

Prepare the text data for the computer (change words into numbers).

Train a classification model to recognize spam patterns.

Test the model and build a simple interface to check custom messages.

5. Methodology / Tools and Technologies Used

To build our project, we used the following tools:

Python: The main programming language because it is great for data science.

Pandas: A library we used to load and read our dataset from the internet.

Scikit-learn: The most important Machine Learning library in our project. We used it to split data, transform text, and train our model.

Google Colab / VS Code: Used for writing and testing the code quickly.

GitHub: Used as a code repository to save our work.

6. Main Features / Model Description

For our model, we chose the Multinomial Naive Bayes algorithm. This is a very popular and fast mathematical algorithm for text classification.

How it works: it calculates the probability of each word. If the model sees that words like "WIN", "FREE", or "PRIZE" appear very often in spam messages during training, it will flag new messages with these words as spam.

Before giving text to the model, we used a feature called CountVectorizer (Bag-of-Words). It takes all the words from the messages, creates a dictionary, and turns every message into a matrix of numbers.

7. Implementation Explanation

We built the project step by step:

Data Loading: We downloaded the open SMS Spam Collection dataset using a direct link. We converted the labels: 'ham' became 0, and 'spam' became 1.

Data Splitting: We used `train_test_split` to divide our data. 80% was used to train the model, and 20% was hidden to test the model later.

Vectorization: We applied `CountVectorizer` to transform our text data into numbers.

Training: We created the `MultinomialNB()` object and trained it using our 80% data.

Interactive Menu: We added a `while True` loop so the user can type their own messages from the keyboard and get the result immediately.

8. Screenshots

Screenshot 1: [

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

def main():
    print("Loading data...")
    link = "https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/data/sms.tsv"
    tablica = pd.read_table(link, header=None, names=['label', 'message'])

    tablica['label_num'] = tablica['label'].map({'ham': 0, 'spam': 1})

    tekst = tablica['message']
    metki = tablica['label_num']

    xTrain, xTest, yTrain, yTest = train_test_split(tekst, metki, test_size=0.2, random_state=42)

    vect = CountVectorizer()
    xTrainDtm = vect.fit_transform(xTrain)
    xTestDtm = vect.transform(xTest)

    model = MultinomialNB()
    model.fit(xTrainDtm, yTrain)

    print("Model ready.\n")

    yPred = model.predict(xTestDtm)
    acc = accuracy_score(yTest, yPred)

    yPred = model.predict(xTestDtm)
    acc = accuracy_score(yTest, yPred)
    print("Accuracy is: " + str(round(acc * 100, 2)) + "%\n")

    print("--- AUTOMATIC TEST ---")
    massiv = [
        "WINNER!! You have been selected to receive a $1000 prize. Call now!",
        "Hey bro, are we still meeting for lunch today?",
        "URGENT: Your bank account has been locked. Click here to verify."
    ]

    testDtm = vect.transform(massiv)
    preds = model.predict(testDtm)

    for i in range(len(massiv)):
        if preds[i] == 1:
            print("[SPAM] " + massiv[i])
        else:
            print("[NOT SPAM] " + massiv[i])

    print("\n--- MANUAL TEST ---")
    print("Type 'exit' to stop.")

    while True:
        msg = input("\nEnter message: ")

        if msg.strip().lower() == 'exit':
            print("Stopping...")
            break
```

```

        if msg.strip().lower() == 'exit':
            print("Stopping...")
            break

        userDtm = vect.transform([msg])
        if model.predict(userDtm)[0] == 1:
            print("=> Result: [SPAM]")
        else:
            print("=> Result: [NOT SPAM]")

    if __name__ == "__main__":
        main()

```

```

... Loading data...
Model ready.

```

```

Accuracy is: 99.19%

```

]

Screenshot 2: [

```

... Loading data...
Model ready.

```

```

Accuracy is: 99.19%

```

```

--- AUTOMATIC TEST ---

```

```

[SPAM] WINNER!! You have been selected to receive a $1000 prize. Call now!

```

```

[NOT SPAM] Hey bro, are we still meeting for lunch today?

```

```

[SPAM] URGENT: Your bank account has been locked. Click here to verify.

```

```

--- MANUAL TEST ---

```

```

Type 'exit' to stop.

```

```

Enter message: hello mom

```

```

=> Result: [NOT SPAM]

```

```

Enter message: YOU HAVE A PRIZE $1000. Call NOW!

```

```

=> Result: [SPAM]

```

```

Enter message: 

```

]

9. Testing and Evaluation Results

We tested our model on the 20% of data it had never seen before. The results were excellent:

Accuracy: Our model achieved an accuracy of 99.19%. This means it guesses correctly 99 times out of 100.

Automatic Test: We wrote a script to test 3 prepared sentences. The model correctly identified the winner-prize text as [SPAM] and the lunch text as [NOT SPAM].

Manual Test: We also typed custom messages during our demo video, and the model reacted perfectly to most common spam phrases.

10. Difficulties and Solutions

During the project, we found two interesting problems:

False Negatives with new words: During our manual test, we typed: "YOU WIN 1000 \$, send message yes..." but the model said it was NOT SPAM.

Solution / Analysis: We realized our dataset is mostly British SMS from the past. The model knows the "£" sign and the word "txt", but it didn't learn the word "CASINO" or "\$". To fix this in the future, we need to update our dataset with modern and international spam examples.

GitHub Autograder Error: We got a red cross in GitHub Actions with the error: "The job was not started because recent account payments have failed".

Solution: We understood this was a server-side problem with GitHub billing, not a bug in our code. We verified that our code works perfectly on local machines and recorded a demo video to prove it.

11. Conclusion

In conclusion, we successfully created a working Machine Learning project. We learned how to process text data, how to train a Naive Bayes classifier, and how to evaluate its accuracy. Despite a few vocabulary limitations in the old dataset, the model works extremely fast and shows a very high accuracy of 99.19%. This project was a great practical experience in Python and Data Science.

12. GitHub and Demo Video Links

GitHub Repository Link: [<https://github.com/kairzhanovgibrat-cmd/Educational-Practice>]

Demo Video Link:

[<https://drive.google.com/file/d/1pxoTNO3zmWeiiud4zyu0X9AaVKeCwC6W/view?usp=sharing>]